

Aaditya Dawar

1024150093

2013

LAB ASSIGNMENT -2

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members(properties):

Name

Roll No

Degree

Hostel

Current CGPA

Member Function(behavior):

addDetails();

updateDetails();

updateCGPA();

updateHostel();

displaydetails();

```
#include <iostream>
```

```
#include <string>
```

```
struct Student {
```

```
    std::string name;
```

```
    int rollNo;
```

```
    std::string degree;
```

```
    std::string hostel;
```

```
    float cgpa;
```

```
void addDetails() {
```

```
    std::cout << "Enter Name: ";
```

```
std::getline(std::cin >> std::ws,  
name);
```

```
std::cout << "Enter Roll No: ";
```

```
std::cin >> rollNo;
```

```
std::cout << "Enter Degree: ";
```

```
std::getline(std::cin >> std::ws,  
degree);
```

```
std::cout << "Enter Hostel: ";
```

```
std::getline(std::cin >> std::ws,  
hostel);
```

```
std::cout << "Enter CGPA: ";
```

```
std::cin >> cgpa;
```

```
}
```

```
void updateCGPA() {
```

```
std::cout << "Enter new CGPA: ";
```

```
std::cin >> cgpa;
```

```
}
```

```
void updateHostel() {
```

```
std::cout << "Enter new Hostel: ";
```

```
std::getline(std::cin >> std::ws,  
hostel);
```

```
}
```

```
void updateDetails() {
```

```

        std::cout << "\n--- Updating All
Details ---" << std::endl;

        addDetails();

    }

    void displayDetails() {

        std::cout << "\n----- Student Record
-----" << std::endl;

        std::cout << "Name:  " << name <<
std::endl;

        std::cout << "Roll No: " << rollNo <<
std::endl;

        std::cout << "Degree: " << degree <<
std::endl;

        std::cout << "Hostel: " << hostel <<
std::endl;

        std::cout << "CGPA:  " << cgpa <<
std::endl;

        std::cout << "-----
-" << std::endl;

    }

};

int main() {

    Student s1;

    std::cout << "Initial Data Entry:" <<
std::endl;

    s1.addDetails();

```

```
s1.displayDetails();
```

```
std::cout << "\nUpdating Specific  
Fields..." << std::endl;
```

```
s1.updateCGPA();
```

```
s1.updateHostel();
```

```
std::cout << "\nFinal Record:" <<  
std::endl;
```

```
s1.displayDetails();
```

```
return 0;
```

```
}
```

```

Initial Data Entry:
Enter Name: Aaditya
Enter Roll No: 1024150093
Enter Degree: be
Enter Hostel: d
Enter CGPA: 8

----- Student Record -----
Name:    Aaditya
Roll No: 1024150093
Degree:  be
Hostel:  d
CGPA:    8
-----

Updating Specific Fields...
Enter new CGPA: 9
Enter new Hostel: O

Final Record:

----- Student Record -----
Name:    Aaditya
Roll No: 1024150093
Degree:  be
Hostel:  O
CGPA:    9
-----

```

2. Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public scope.

```
#include <iostream>
```

```
#include <string>
```

```
class Student {
```

```
private:
```

```
    // Private Data Members
```

```
    std::string name;
```

```
    int rollNo;
```

```
    std::string degree;
```

```
    std::string hostel;
```

```
    float cgpa;
```

```
    // Private Member Function (Internal Logic)
```

```
    void validateAndSetCGPA(float newCgpa) {
```

```
        if (newCgpa >= 0.0 && newCgpa <= 10.0) {
```

```
            cgpa = newCgpa;
```

```

    } else {
        std::cout << "Invalid CGPA! Value remains unchanged." << std::endl;
    }
}

public:
// Public Member Functions
void addDetails() {
    std::cout << "Enter Name: ";
    std::getline(std::cin >> std::ws, name);
    std::cout << "Enter Roll No: ";
    std::cin >> rollNo;
    std::cout << "Enter Degree: ";
    std::getline(std::cin >> std::ws, degree);
    std::cout << "Enter Hostel: ";
    std::getline(std::cin >> std::ws, hostel);

    float tempCgpa;
    std::cout << "Enter CGPA: ";
    std::cin >> tempCgpa;
    validateAndSetCGPA(tempCgpa);
}

void updateCGPA() {
    float newCgpa;
    std::cout << "Enter new CGPA: ";
    std::cin >> newCgpa;
    validateAndSetCGPA(newCgpa); // Accessing private function from public scope
}

void updateHostel() {
    std::cout << "Enter new Hostel: ";
    std::getline(std::cin >> std::ws, hostel);
}

void displayDetails() {
    std::cout << "\n----- Student Record (Class) -----" << std::endl;
    std::cout << "Name:  " << name << "\nRoll No: " << rollNo
        << "\nDegree:  " << degree << "\nHostel:  " << hostel
        << "\nCGPA:   " << cgpa << std::endl;
    std::cout << "-----" << std::endl;
}
};

```

```

int main() {
    Student s1;

    // Direct access to s1.name or s1.cgpa here would cause a compile error
    s1.addDetails();
    s1.displayDetails();

    std::cout << "\nModifying specific details..." << std::endl;
    s1.updateCGPA();
    s1.displayDetails();

    return 0;
}

```

```

Enter Name: Aaditya
Enter Roll No: 1024150093
Enter Degree: be
Enter Hostel: D
Enter CGPA: 8

----- Student Record (Class) -----
Name:    Aaditya
Roll No: 1024150093
Degree:  be
Hostel:  D
CGPA:    8
-----

Modifying specific details...
Enter new CGPA: 9

----- Student Record (Class) -----
Name:    Aaditya
Roll No: 1024150093
Degree:  be
Hostel:  D
CGPA:    9
-----

...Program finished with exit code 0
Press ENTER to exit console.

```

3. Create a code snippet that illustrates the following: Calling of private member functions inside public member function

```
#include <iostream>
```

```

#include <string>

class SecureSystem {
private:
    std::string secretKey = "Admin123";

    bool validateKey(std::string inputKey) {
        return inputKey == secretKey;
    }

public:
    void accessData() {
        std::string userKey;
        std::cout << "Enter access key: ";
        std::cin >> userKey;

        if (validateKey(userKey)) {
            std::cout << "Access Granted. Confidential data: [42.09, 88.12]" << std::endl;
        } else {
            std::cout << "Access Denied." << std::endl;
        }
    }
};

int main() {
    SecureSystem sys;
    sys.accessData();
    return 0;
}

```

```

Enter access key: Admin123
Access Granted. Confidential data: [42.09, 88.12]

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Define a class Rectangle with variables width and height of integer typwe along with following:

- (a) void getdata() to initialize object values
- (b) void calculatearea() to calculate and display the area.

```

#include <iostream>

```

```

class Rectangle {
private:
    int width;
    int height;

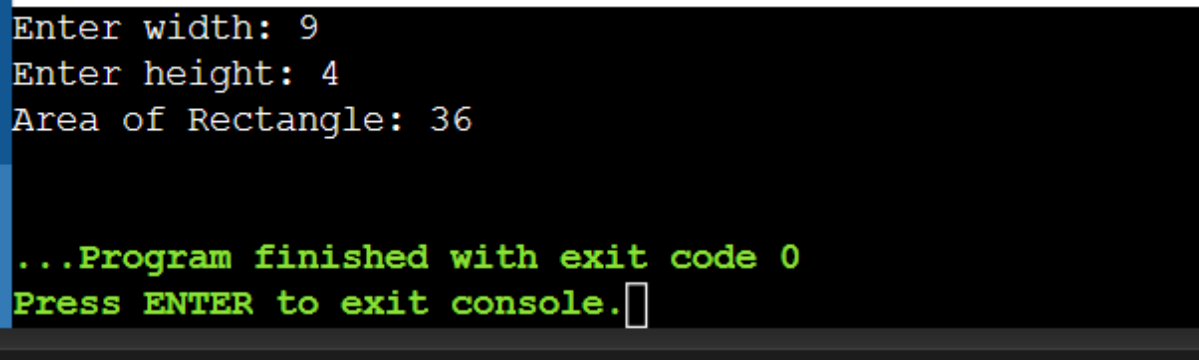
public:
    void getdata() {
        std::cout << "Enter width: ";
        std::cin >> width;
        std::cout << "Enter height: ";
        std::cin >> height;
    }

    void calculatearea() {
        int area = width * height;
        std::cout << "Area of Rectangle: " << area << std::endl;
    }
};

int main() {
    Rectangle rect;
    rect.getdata();
    rect.calculatearea();
    return 0;
}

```

OUTPUT-



```

Enter width: 9
Enter height: 4
Area of Rectangle: 36

...Program finished with exit code 0
Press ENTER to exit console.

```

5. Define a class Complex with variables real and imaginary along with following:

- (a) void setComplex (float, float) to initialize object values.
- (b) void displayComplex() to show the complex number
- (c) Pass and return objects to calculate sum of two complex numbers. Display the sum.

```
#include <iostream>
```

```

class Complex {
private:
    float real;
    float imaginary;

public:
    void setComplex(float r, float i) {
        real = r;
        imaginary = i;
    }

    void displayComplex() {
        std::cout << real << " + " << imaginary << "i" << std::endl;
    }

    Complex addComplex(Complex c) {
        Complex temp;
        temp.real = real + c.real;
        temp.imaginary = imaginary + c.imaginary;
        return temp;
    }
};

int main() {
    Complex c1, c2, sum;
    float r1, i1, r2, i2;

    std::cout << "Enter real and imaginary part for first number: ";
    std::cin >> r1 >> i1;
    c1.setComplex(r1, i1);

    std::cout << "Enter real and imaginary part for second number: ";
    std::cin >> r2 >> i2;
    c2.setComplex(r2, i2);

    sum = c1.addComplex(c2);

    std::cout << "First Complex Number: ";
    c1.displayComplex();
    std::cout << "Second Complex Number: ";
    c2.displayComplex();
    std::cout << "Sum: ";
    sum.displayComplex();

    return 0;
}

```

```

Enter real and imaginary part for first number: 8 0
Enter real and imaginary part for second number: 9 1
First Complex Number: 8 + 0i
Second Complex Number: 9 + 1i
Sum: 17 + 1i

...Program finished with exit code 0
Press ENTER to exit console.

```

6. Implement *scope resolution operator* :: for the following uses:

- (a) Class functions defined outside the class
- (b) Access a global variable with same name as a local variable
- (c) Access a static variables
- (d) Use inbuilt libraries (cin and cout with scope resolution operator)

```

#include <iostream>
#include <string>

int main() {
    int age;
    double salary;
    std::string name;

    std::cout << "Enter your full name: ";
    std::getline(std::cin, name);

    std::cout << "Enter your age: ";
    std::cin >> age;

    std::cout << "Enter your expected salary: ";
    std::cin >> salary;

    std::cout << "\n--- User Profile ---" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Age: " << age << std::endl;
    std::cout << "Salary: " << salary << std::endl;

    return 0;
}

```

OUTPUT-

```

Enter your full name: Aaditya
Enter your age: 20
Enter your expected salary: 60000

--- User Profile ---
Name: Aaditya
Age: 20
Salary: 60000

```

7. Create a code to implement the *namespace* and use similar variables and functions

defined in different code sections.

```
#include <iostream>
#include <string>

namespace Sales {
    int quota = 5000;
    void displayStatus() {
        std::cout << "Sales Quota: " << quota << std::endl;
    }
}

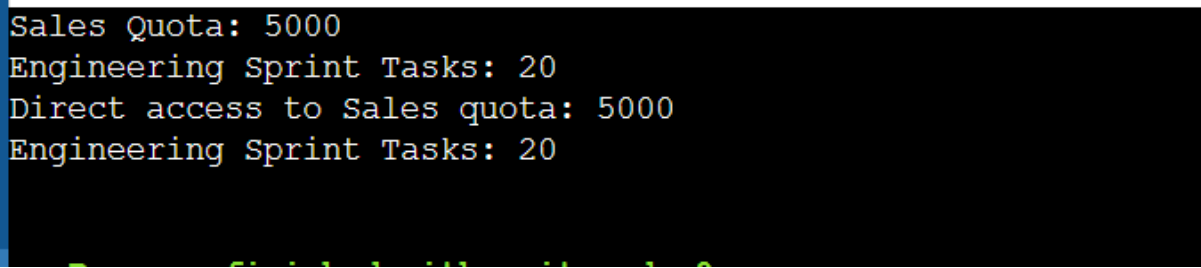
namespace Engineering {
    int quota = 20;
    void displayStatus() {
        std::cout << "Engineering Sprint Tasks: " << quota << std::endl;
    }
}

int main() {
    Sales::displayStatus();
    Engineering::displayStatus();

    using Sales::quota;
    std::cout << "Direct access to Sales quota: " << quota << std::endl;

    {
        using namespace Engineering;
        displayStatus();
    }

    return 0;
}
```

A screenshot of a terminal window with a black background and white text. It shows the output of the C++ program: 'Sales Quota: 5000', 'Engineering Sprint Tasks: 20', 'Direct access to Sales quota: 5000', and 'Engineering Sprint Tasks: 20'.

```
Sales Quota: 5000
Engineering Sprint Tasks: 20
Direct access to Sales quota: 5000
Engineering Sprint Tasks: 20
```